CHAPTER 38

Transports and the Protocol Lifecycle

In this chapter

Introduction: The Refund That Happened Twice . . . 3
38.1 Core Concepts . . . 4
38.2 Three Message Shapes and Their Rules . . . 5
38.3 STDIO: The Wire Is stdout . . . 6
38.4 Streamable HTTP: One Endpoint, Two Methods . . . 6
38.5 Sessions . . . 7
38.6 Resumability and Redelivery . . . 8
38.7 The Lifecycle . . . 8
38.8 Cancellation Is Not a Retry Licence . . . 9
38.9 Timeouts, and the Maximum That Is Only a SHOULD . . . 9
38.10 The Protocol Version Header . . . 10
38.11 Backwards Compatibility With HTTP+SSE . . . 10
38.12 Analogies, and Where They Break . . . 11
38.13 Architecture . . . 11
38.14 Worked Example: Making One Tool Call Safely . . . 12
38.15 Code Walkthrough: A Transport You Can Break On Purpose . . . 13
38.16 Mini Project: Breaking the Transport on Purpose . . . 14
38.17 STDIO: One Skipped Line, One Lost Answer . . . 15
38.18 Streamable HTTP: Fifteen Outcomes, No Socket . . . 16
38.19 Lifecycle Ordering: Ten Rules . . . 16
38.20 Resumability: Fourteen Cut Points . . . 17
38.21 Disconnection Is Not Cancellation . . . 18
38.22 Timeouts, and the Pool . . . 18
38.23 Hands-on Labs . . . 19
38.24 Lab Solutions . . . 19
38.25 Best Practices . . . 20
38.26 Common Mistakes . . . 21
38.27 Performance Considerations . . . 22
38.28 Security Considerations . . . 23

38.29 Real Production Examples 23
38.30 Certification Notes 24
38.31 Twenty Practice Questions 25
38.32 Answers 25
38.33 Ten Practical Exercises 26
38.34 Five Challenge Labs 27
38.35 Ten Interview Questions 27
38.36 Summary 28
38.37 Glossary 29
38.38 Further Reading 30

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Frame and parse MCP's three JSON-RPC message shapes — request, response and notification — and say which fields each one must and must not carry.
2. Operate the **STDIO** transport correctly: one message per line, nothing but messages on `stdout`, and logs on `stderr` where they belong.
3. Operate the **Streamable HTTP** transport: one endpoint, POST and GET, the `Accept` requirements, and every documented status code including 202, 400, 403, 404 and 405.
4. Manage a session end to end — `MCP-Session-Id`, termination, the 404 that means *start again*, and `DELETE` — and explain why a session ID is not an authentication token.
5. Resume a broken stream with `Last-Event-ID`, state the per-stream replay rule, and identify the one cut point from which no resumption is possible.
6. Explain why **a disconnection is not a cancellation**, and measure what happens to effects when a client assumes otherwise.
7. Send and handle `notifications/cancelled` correctly, including the race conditions the specification says both parties must handle gracefully.
8. Set timeouts that terminate: reset on progress if you like, and always enforce a maximum, with the pool arithmetic to justify the number.

NOTE

Prerequisites

Chapter 17 for server-sent events, cumulative deltas, and the truncation harness that cut a stream at 93 points — the instrument this chapter reuses. **Chapter 29** for retryable and non-retryable outcomes, for backoff and jitter, and for the rule that *a client-side timeout does not cancel the work*. **Chapter 34** for idempotency, for the distinction between retrying a call and retrying a turn, and for the 31.65 duplicated effects per 1,000 that turn-layer retry produced. **Chapter 36** for the MCP connector's transport constraint. **Chapter 37** for the host/client/server roles, for capability negotiation, and for the protocol-version header's silent default.

Deferred by name: **resources, tools and prompts as protocol primitives** are **Chapter 39**; **building, securing and shipping a server**, including the authorization framework this chapter only points at, is **Chapter 40**; **MCP inside Claude Code** is **Chapter 41**. **Prompt injection through a tool result** is **Chapter 58**; **observability and tracing across a transport** is **Chapter 57**.

Introduction: The Refund That Happened Twice

An operations team runs an assistant with a payments MCP server behind it. A support agent approves a refund. The tool call goes out over Streamable HTTP, the server starts work, and forty seconds later the connection drops — a load balancer idle timeout, the kind of thing that happens a few hundred times a day in any real network.

The client does what looks obviously correct. The stream died, so the request must have died with it; the request must be reissued. It builds a fresh JSON-RPC id and sends `tools/call` again.

The customer receives two refunds.

Nothing in that story is a bug in the sense of a mistake in a line of code. The client's reasoning was wrong at the level of the protocol, and the specification says so in one sentence: *"Disconnection SHOULD NOT be interpreted as the client cancelling its request."* A dropped TCP connection is a fact about a socket. The server's work is a fact about the server, and the two are unrelated. Chapter 29 established the same thing for the Messages API — *a client-side timeout does not cancel the generation* — and Chapter 34 measured what happens when a retry crosses a side effect. This chapter measures it again at the transport layer, and the number is worse: **210 duplicated effects per 1,000 calls** against Chapter 34's 31.65 at the turn layer, because a transport-layer retry fires on every network hiccup rather than only on an error the application saw.

MCP gives you two instruments the Messages API does not. It defines an explicit **cancellation notification**, so a client can genuinely ask a server to stop. And it defines **resumability**: attach an id to each server-sent event, and a client that reconnects with `Last-Event-ID` can pick the stream up where it broke rather than starting again.

Both help. Neither is a licence to retry. §38.21 measures all four policies over the same 1,000 calls: naive reissue at 210 per 1,000, explicit cancellation followed by a retry at **67**, resumption at **17**, and only Chapter 34's idempotent effect at **0.00**. The cancellation result is the one worth sitting with. Cancellation is a *notification* — fire and forget — that *"may arrive after request processing has completed"*, and receivers *"MAY ignore"* it. Asking politely for something to stop is not the same as it having stopped.

This chapter is about the layer where those guarantees live: how bytes are framed, how sessions begin and end, what a broken connection actually means, and what you are entitled to conclude from silence. It is the least glamorous chapter in Part 7 and it is the one whose mistakes cost money.

38.1 Core Concepts

DEFINITION

Transport

The mechanism carrying JSON-RPC messages between a client and a server. MCP defines two: **stdio** and **Streamable HTTP**. Clients *"SHOULD support stdio whenever possible"*. Custom transports are permitted provided they preserve the message format and lifecycle.

DEFINITION

JSON-RPC 2.0

The message format. All MCP messages **MUST** follow it, and **MUST** be UTF-8 encoded. Three shapes only: request, response and notification.

DEFINITION

Framing

How a receiver knows where one message ends and the next begins. Over stdio it is a newline; over HTTP it is the request or response body, or an SSE event.

DEFINITION

Session

One logically related run of interactions beginning with initialization. Over stdio it is the lifetime of the subprocess. Over Streamable HTTP a server **MAY** assign an `MCP-Session-Id` at initialization, in which case the client **MUST** send it on every subsequent request.

DEFINITION

Server-sent events (SSE)

A one-way stream of text events over a single HTTP response, the same mechanism Chapter 17 used for the Messages API's `stream: true`. Under Streamable HTTP a POST or a GET may open one.

DEFINITION

Event id

An optional `id` field on an SSE event. If present it **MUST** be globally unique across all streams within the session, and it **SHOULD** encode enough information to identify its own stream. It is the handle a client resumes from.

DEFINITION

Resumption

Reconnecting with a `Last-Event-ID` header so the server can replay what was missed — *"on the stream that was disconnected"*, and never on another one.

DEFINITION

Cancellation notification

`notifications/cancelled`, carrying a `requestId` and an optional `reason`. A notification, so it has no id and receives no response. It is a request to stop, not a guarantee of having stopped.

DEFINITION

Progress notification

`notifications/progress`, carrying a `progressToken` that the *requester* placed in `params._meta`, a monotonically increasing `progress`, and optional `total` and `message`. It exists so a long operation can prove it is alive.

38.2 Three Message Shapes and Their Rules

Everything on an MCP wire is one of three things, and each has rules with teeth.

A **request** carries `jsonrpc`, an `id` and a `method`. The id **MUST** be a string or an integer, it **MUST NOT** be `null` — a departure from base JSON-RPC, where null is legal — and it **MUST NOT** have been used before

by the same requestor within the same session. That last rule is the one that catches people writing retry logic: reusing an id to "try again" is a protocol violation, and minting a new one is a new request that the server has no reason to associate with the old one.

A **response** carries the same `id` and exactly one of `result` or `error`. An error response carries `code`, `message` and optional `data`, and the code **MUST** be an integer.

A **notification** carries `jsonrpc` and `method` and **MUST NOT** carry an id. The receiver **MUST NOT** respond. `notifications/initialized`, `notifications/cancelled`, `notifications/progress` and the various `list_changed` messages are all notifications, and the absence of an id is the whole reason cancellation cannot be confirmed.

KEY TAKEAWAYS

The shape decides the guarantee

A request gets an answer. A notification does not, and cannot. When you find yourself wanting to know whether a notification was honoured, you have found a place where the protocol will not tell you and your design has to cope.

38.3 STDIO: The Wire Is `stdout`

Over stdio, *"the client launches the MCP server as a subprocess"*. Messages go in on `stdin` and out on `stdout`, one per line, newline-delimited, and — the sentence that causes more lost afternoons than any other in the specification — *"The server **MUST NOT** write anything to its `stdout` that is not a valid MCP message."*

Every language's default debugging instinct violates this. `print()`. `console.log()`. A package manager's install notice. A progress bar. A framework's deprecation warning. All of them land on `stdout`, all of them are on the wire, and none of them is a message.

The specification provides the correct channel and is careful about how to read it: *"The server MAY write UTF-8 strings to its standard error (`stderr`) for any logging purposes"*, and the client *"MAY capture, forward, or ignore"* it and *"**SHOULD NOT** assume `stderr` output indicates error conditions"*. A line on `stderr` is not a failure. It is a log line that had nowhere else to go.

The second rule is that messages *"**MUST NOT** contain embedded newlines"*. This one is usually satisfied for free, and it is worth understanding why, because the exception is where the damage is. A newline inside a *string value* is escaped to `\n` by any JSON encoder, so ordinary content can never break framing. The rule bites when something bypasses the encoder — a hand-built line, or, far more commonly, a pretty-printer. §38.17 puts an eight-message session through `json.dumps(msg, indent=2)` and recovers **0 messages from 57 lines**. Valid JSON, every one. Unreadable as a transport.

WARNING

The failure mode is not a crash

A polluted `stdout` does not produce an exception in the server. It produces a client that cannot parse a line, and what the client does next decides everything. §38.17 measures both choices.

38.4 Streamable HTTP: One Endpoint, Two Methods

The server *"MUST provide a single HTTP endpoint path … that supports both POST and GET methods"* — one URL, for example `https://example.com/mcp`.

Every message the client sends is a new HTTP POST. The client **MUST** include an `Accept` header listing *both* `application/json` and `text/event-stream`, because it does not get to choose which it receives. The body **MUST** be a single JSON-RPC request, notification or response.

What comes back depends on what went out:

- If the body was a **notification or a response**, and the server accepts it, the server **MUST** return **202 Accepted with no body**. There is nothing to answer.
- If the body was a **request**, the server **MUST** return either `Content-Type: application/json` with one JSON object, or `Content-Type: text/event-stream` to open an SSE stream — and *"The client MUST support both these cases."* You do not get to insist on one.

When the server opens a stream it **MAY** send requests and notifications of its own before the response, it **SHOULD** eventually include the response for the request that opened it, and it **SHOULD** terminate the stream afterwards. It may also close the *connection* without terminating the *stream*, having sent a `retry` field the client **MUST** respect — which is how a server avoids holding thousands of long-lived sockets and turns them into polite polling instead.

The client MAY also issue a GET to open a stream with no request attached, so the server can push notifications and its own requests. Here the `Accept` header **MUST** list `text/event-stream`, and the server either returns one or returns **405 Method Not Allowed** to say it does not offer one at this endpoint.

One rule about multiple streams matters more than it looks: the server *"**MUST** send each of its JSON-RPC messages on only one of the connected streams; that is, it **MUST NOT** broadcast the same message across multiple streams."* A client holding two streams sees each message once, on one of them. Any de-duplication logic you write on the assumption of broadcast will silently discard distinct messages that happen to look alike.

38.5 Sessions

A server **MAY** assign a session at initialization by returning an `MCP-Session-Id` header on the response containing the `InitializeResult`. The id **SHOULD** be globally unique and cryptographically secure — *"a securely generated UUID, a JWT, or a cryptographic hash"* — and **MUST** contain only visible ASCII, `0x21` to `0x7E`.

From then on the rules are mechanical and each has a status code attached.

Situation	Documented outcome
Client omits the id on a non-initialization request	**400** Bad Request (for servers that require one)
Server has terminated the session	**404** Not Found
Client receives that 404	**MUST** start a new session with a fresh `InitializeRequest` and no id

Situation	Documented outcome
Client sends `DELETE` with the id	Session terminated
Server does not permit client termination	**405** Method Not Allowed

The 404 rule is the one to internalise, because it is not an error to report — it is an instruction. A 404 on a session means *begin again*, and a client that surfaces it to a user as a failure has turned a recoverable condition into an outage.

> WARNING
>
> **A session ID is not a credential**
>
> The security guidance is unambiguous: MCP servers implementing authorization **MUST** verify all inbound requests, and **MUST NOT** use sessions for authentication. Session IDs **MUST** be non-deterministic, and servers **SHOULD** bind them to user-specific information — a key of the form `<user_id>:<session_id>` — so that guessing an id does not let an attacker act as someone else. Two attacks are documented in detail: hijack-by-impersonation, and a hijack that injects a malicious event into a shared queue which the legitimate client then receives as a resumed response.

38.6 Resumability and Redelivery

This is the part of the transport that exists specifically so that a broken connection does not have to become a repeated action.

The mechanism is small. Servers **MAY** attach an `id` to their SSE events. If they do, the id **MUST** be globally unique across every stream in the session, and it **SHOULD** encode enough information to identify the originating stream. A client that disconnects **SHOULD** issue a GET carrying `Last-Event-ID`, and the server **MAY** replay what came after it — *"on the stream that was disconnected"*. Then the hard limit: the server **MUST NOT** replay messages that would have been delivered on a different stream.

There is one more line, easy to skim and load-bearing: the server *"SHOULD immediately send an SSE event consisting of an event ID and an empty `data` field in order to prime the client to reconnect"*. A stream with no priming event has a window at the start during which the client has nothing to resume from.

§38.20 cuts a twelve-event stream at all fourteen boundaries and measures exactly that. With ids attached, **13 of 14 cut points recover every message**. The one that does not is the cut before any event has arrived: `Last-Event-ID` is absent, there is nothing to resume from, and all twelve messages are lost. Without ids, **1 of 14**. Resuming with an id belonging to another stream is refused at **13 of 14** cut points, exactly as the MUST NOT requires.

That single unrecoverable cut is not a flaw in the harness. It is the residue that resumption cannot reach, and it is why §38.21's `resume` policy still shows a non-zero duplicate rate.

38.7 The Lifecycle

Three phases: initialization, operation, shutdown.

Initialization MUST be first. The client sends `initialize` with a protocol version, its capabilities and its implementation info. The server responds with its own. The client then sends `notifications/initialized`. Two ordering rules sit either side of that exchange, both SHOULD NOTs with real consequences: the client *"SHOULD NOT send requests other than pings before the server has responded"*, and the server *"SHOULD NOT send requests other than pings and logging before receiving the* `initialized` *notification"*. §38.19 reproduces ten ordering rules offline, including the two exceptions — a ping may cross an outstanding `initialize`, and so may a log message from the server.

Operation is bounded by what was negotiated. Chapter 37 covered that rule and this chapter inherits it unchanged.

Shutdown has no messages at all. *"No specific shutdown messages are defined—instead, the underlying transport mechanism should be used to signal connection termination."* Over stdio the client **SHOULD** close the child's `stdin`, wait, then `SIGTERM`, then `SIGKILL` — in that order, with a reasonable interval between each. Over HTTP it is closing the connection, or a `DELETE` to end the session.

That escalation ladder is worth copying verbatim into your own supervisor. A server that is mid-write when you `SIGKILL` it leaves whatever it was doing half-done, and the protocol will not tell you which half.

38.8 Cancellation Is Not a Retry Licence

`notifications/cancelled` carries the `requestId` and an optional `reason`. Everything else about it is a set of qualifications:

- It **MUST** only reference a request issued in the same direction and believed still in progress.
- The `initialize` request **MUST NOT** be cancelled by clients.
- Receivers **SHOULD** stop processing, free resources, and **not** send a response.
- Receivers **MAY** ignore it if the request is unknown, already complete, or not cancellable.
- The sender **SHOULD** ignore any response that arrives afterwards.
- *"Due to network latency, cancellation notifications may arrive after request processing has completed, and potentially after a response has already been sent."* Both parties **MUST** handle that race gracefully.

Read the list as a whole and the conclusion is inescapable. You can ask a server to stop. You cannot learn that it did. There is no acknowledgement, because a notification has no id, and there are three documented reasons a conforming server may do nothing at all.

So cancellation is the right thing to send when a user presses stop — it frees the server's resources in the common case, and the specification says to log the reason. It is not evidence on which to base a retry that has a side effect. §38.21 measures the gap: cancel-then-retry still duplicated **67 effects per 1,000**.

The remedy is not in this chapter. It is Chapter 34's, unchanged: **make the effect idempotent, with a key minted by your loop, upstream of the model and upstream of the transport.** Then it does not matter whether the cancellation arrived, whether the stream resumed, or how many times the network coughed.

38.9 Timeouts, and the Maximum That Is Only a SHOULD

Implementations *"SHOULD establish timeouts for all sent requests, to prevent hung connections and resource exhaustion"*, and on expiry the sender *"SHOULD issue a cancellation notification for that request and stop waiting"*.

Then a refinement, and then a warning about the refinement. Implementations *"MAY choose to reset the timeout clock when receiving a progress notification … as this implies that work is actually happening"*. But: *"implementations SHOULD always enforce a maximum timeout, regardless of progress notifications, to limit the impact of a misbehaving client or server."*

A reset-on-progress timeout with no maximum is not a long timeout. It is no timeout. Any server emitting progress more often than the idle interval holds the request open indefinitely, and it does not have to be malicious to do it — a stuck loop that still ticks a counter is enough. §38.22 simulates a thirty-second idle timeout against progress every five seconds and reports that **no timeout fires in twelve hours of simulated time**. With a maximum of 300 seconds it fires at 300.

The cost is not abstract. Each hung request holds a session slot. At four hung requests a minute against a pool of 32, no maximum exhausts the pool in **8 minutes**; a 300-second maximum settles at **20 of 32 slots** held and never exhausts it.

> KEY TAKEAWAYS
>
> **Two numbers, not one**
>
> An idle timeout says "nothing has happened lately". A maximum says "this has gone on long enough". You need both, and only one of them is in most client libraries by default.

38.10 The Protocol Version Header

Over HTTP the client **MUST** include `MCP-Protocol-Version: <negotiated-version>` on every request after initialization. Two rules follow.

If the header is **missing** and the server has no other way to identify the version, it *"SHOULD assume protocol version* `2025-03-26`". Nothing errors. Capabilities added since simply are not there, and the failure looks like a feature that works over stdio and not over HTTP.

If the header is **present and invalid or unsupported**, the server **MUST** respond **400 Bad Request**. A wrong version is an error; a missing one is a silent downgrade. That asymmetry is the reverse of most people's intuition and it is worth knowing before you debug it.

Chapter 37 introduced this alongside `anthropic-version` from Chapter 26. The pattern is the same in all three places: declare the contract you are speaking, and the other side is free to grow without breaking you.

38.11 Backwards Compatibility With HTTP+SSE

Streamable HTTP replaced the HTTP+SSE transport of revision 2024-11-05, and the specification documents how each side keeps talking to the other.

Servers wanting to support older clients continue to host the old SSE and POST endpoints alongside the new MCP endpoint.

Clients wanting to support older servers perform a documented probe: POST an `InitializeRequest` to the URL with the correct `Accept` header. If it succeeds, the server speaks Streamable HTTP. If it fails with **400, 404 or 405**, issue a GET, expect an SSE stream whose first event is an `endpoint` event, and use the old transport from then on.

Note which status codes are in that list. A 404 during the probe means *this might be an old server*, and a 404 on an established session means *your session is gone, start again*. Same code, two meanings, distinguished only by where you are in the lifecycle.

38.12 Analogies, and Where They Break

VISUAL ANALOGY

The phone call and the paperwork

Sending a request is telephoning a colleague and asking them to process a refund. The line going dead tells you nothing about whether they processed it. Ringing back and asking again is how the customer gets two refunds.

Cancellation is ringing back and saying "actually, stop" — helpful, usually effective, and unconfirmed. They may already have finished. They may not have picked up. You will not be told.

Where it breaks. A colleague would notice a second identical request and query it. A server has no memory across requests unless you gave it one — which is exactly what an idempotency key is.

VISUAL ANALOGY

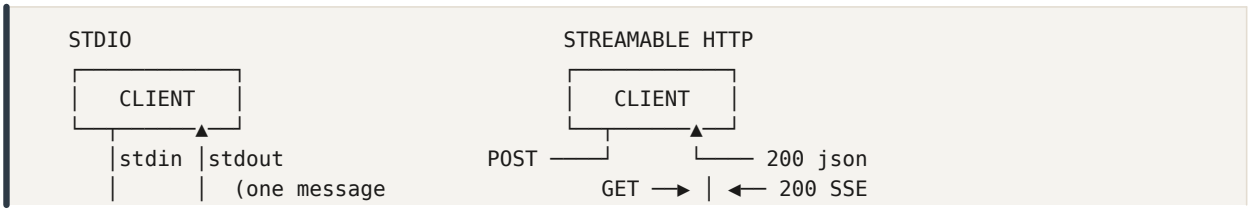
The conveyor belt with numbered parcels

An SSE stream is a belt of numbered parcels. If the belt stops, you tell the depot the number of the last parcel you received and they restart from the next one. That is `Last-Event-ID`, and it works precisely because the parcels are numbered.

Two things break it. If the depot never numbered them, there is nothing to say. And if you quote a number from a *different* belt, the depot must refuse — not out of pedantry, but because the numbers only mean anything within one belt.

Where it breaks. A real depot would ship you a replacement parcel. A server replaying a stream is not re-doing the work; it is re-sending messages it already generated. If the work itself did not happen, no amount of replay creates it.

38.13 Architecture



```
    │      │   per line)                    DELETE ▶│ ◀── 202 / 400
┌───▼──────┴────┐                          ┌─────▼──────┐  404 / 405
│   SERVER      │──▶ stderr (logs)         │   SERVER   │  403
│ subprocess    │    NOT the wire          │  one URL,  │
└───────────────┘                          │  POST+GET  │
                                           └────────────┘
session = the process lifetime         session = MCP-Session-Id
shutdown = close stdin, SIGTERM,       shutdown = close the
           then SIGKILL                           connection, or DELETE
```

```
server ──▶ id=s1-000  (priming event, empty data)   ◀── SHOULD send this
       ──▶ id=s1-001  notifications/progress  1/12
       ──▶ id=s1-002  notifications/progress  2/12
       ──▶ id=s1-003  ──────────╳ CONNECTION DROPS
WRONG                              RIGHT
"the request died"                 "the connection died"
POST tools/call (new id)           GET  Last-Event-ID: s1-002
       │                                  │
       ▼                                  ▼
the server never stopped;          the server replays s1-003 onward
the effect happens TWICE           ON THE SAME STREAM ONLY
210 duplicated effects / 1,000     17 duplicated effects / 1,000
(the residue is the drop that happens before s1-000 arrives)
```

```
                  ┌──────────────────────────────────┐
POST /mcp ──────▶ │ Origin valid?          — no ─▶ 403
Accept: json +    │ MCP-Protocol-Version   — bad ─▶ 400
  event-stream    │   absent → assume 2025-03-26  │
                  │ MCP-Session-Id required? ─────▶ 400 if missing
                  │   known?               — no ─▶ 404  (start again)
                  │ body is a notification/response ▶ 202, no body
                  │ body is a request ──────────────▶ 200 application/json
                  │                            or ─▶ 200 text/event-stream
                  └──────────────────────────────────┘
GET /mcp ──────▶ 200 text/event-stream, or 405 if none is offered
DELETE /mcp ───▶ session ends, or 405 if the server forbids it
```

38.14 Worked Example: Making One Tool Call Safely

A single `tools/call` with a side effect, done properly, is six decisions.

1. Mint the idempotency key before anything else. Chapter 34's rule: minted by your loop, once per logical request, upstream of the model and upstream of the transport. Everything below is damage limitation; this is the actual fix.

2. Send the request with a fresh JSON-RPC id and a `progressToken` in `params._meta` if you want to know the server is alive.

3. Set two timeouts. An idle timeout that may reset on progress, and a maximum that may not. Write both numbers down and justify the second from your pool size.

4. On a disconnection, resume — do not reissue. Reconnect with `Last-Event-ID`. If you have no id because nothing arrived, you are in the one unrecoverable case, and step 1 is what saves you.

5. On a user cancellation, send `notifications/cancelled` with a reason, log it, stop waiting, and ignore any response that arrives later. Do not treat the notification as confirmation of anything.

6. On a 404 for the session, start a new session. Fresh `InitializeRequest`, no session id, then replay your own intent — with the same idempotency key from step 1.

Notice that five of the six steps are about what to do when something goes wrong, and that the one which makes them safe is the one taken before anything has gone wrong at all.

38.15 Code Walkthrough: A Transport You Can Break On Purpose

Four modules, no network, no key. The design rule throughout is that every documented MUST becomes a line that fails, so a test can prove the rule rather than the prose asserting it.

Start with framing, where the interesting part is what the guard does *not* have to catch:

```
def encode(message):
    """One message, one line. Compact separators are not cosmetic.
    Note where the MUST NOT is actually satisfied: a newline inside a
    string is escaped to `\\n` by the JSON encoder, so ordinary content
    can never break framing. The rule bites only when something
    bypasses the encoder - a pretty-printer, or a hand-built line.
    """
    return encode_raw(json.dumps(message, separators=(",", ":")))
```

The first draft put the newline check inside `encode` and a test asserted it fired. It never did, because `json.dumps` had already escaped the newline. That is worth publishing rather than quietly fixing: a guard that cannot fire is not protection, it is decoration, and the way to find out is to write the test that expects it to fire. The guard now lives in `encode_raw`, where a hand-built line actually reaches it.

Next, the two readers. They differ by four lines and by everything:

```
def read_strict(stream):
    """Refuse at the first line that is not a message."""
    ...
        classify(obj)      # raises FramingError
        out.append(obj)
def read_lenient(stream):
    """Skip anything unparseable and carry on. The common shortcut."""
    ...
        except (json.JSONDecodeError, FramingError):
            skipped.append((n, line.rstrip("\n")))
            continue
```

`read_lenient` is what almost every quick implementation does, and it is defensible right up to the moment a skipped line was a *response*. §38.17 measures that case.

The HTTP endpoint is a state machine whose every branch quotes its rule. Here is the session check in full, because all three of its outcomes are documented and all three matter:

```
    def _session(self, headers, initializing=False):
        sid = headers.get("MCP-Session-Id")
```

```
        if initializing:
            return None
        if self.requires_session and sid is None:
            return (400, "missing MCP-Session-Id", None)
        if sid is not None and sid not in self.sessions:
            return (404, "unknown or terminated session", None)
        return None
```

Two things to notice. `initializing` short-circuits, because the id does not exist yet — the initialize request is the one request that legitimately has none. And an unknown id is **404**, not 400 and not 403: it is the code that means *start a new session*, and §38.18 shows a client doing exactly that.

Client-side MUSTs get their own exception type, because they have no status code:

```
        if not ("application/json" in accept
                and "text/event-stream" in accept):
            raise ClientBug("POST Accept MUST list both application/json "
                            "and text/event-stream, got %r" % accept)
```

A malformed `Accept` header is not a server response to handle. It is a request that should never have left your process, and giving it a distinct exception keeps the outcome table honest about which failures are yours.

Finally, the piece that ties two sections together. The residual duplicate rate in §38.21 is not a parameter someone chose; it is read out of the resumability harness:

```
_H = RES.harness()
P_BEFORE_PRIME = 1.0 - (_H["cuts_fully_recovered"] / _H["cut_points"])
```

The harness finds exactly one cut point from which no resumption is possible. That fraction — 1 in 14 — is then the fraction of disconnections a resuming client cannot save. Deriving it rather than assuming it means that if the harness changes, the economics change with it, and no number in the chapter is left stranded.

38.16 Mini Project: Breaking the Transport on Purpose

What it is. A framer and two readers for stdio, a Streamable HTTP endpoint as a state machine, a resumability harness in the shape of Chapter 17's truncation harness, a lifecycle ordering table, and a duplicate-effect simulation over four retry policies.

What it is for. To answer, with numbers rather than intuition, the question this chapter opened with: what may you conclude when a connection drops?

```
   stdioframe.py ──┬─▶ encode / encode_raw / encode_pretty
                   ├─▶ classify()        three JSON-RPC shapes
                   ├─▶ read_strict / read_lenient
                   └─▶ audit()   polluted stdout + indent=2
   httpwire.py ────┬─▶ Endpoint: POST / GET / DELETE
                   ├─▶ Origin · version · session preconditions
                   ├─▶ ClientBug   (a MUST with no status code)
                   └─▶ outcomes()  15 documented results
   resume.py ──────┬─▶ stream() with a priming event and stream-qualified ids
```

```
                    ├─▶ replay()   refuses across streams (MUST NOT)
                    ├─▶ harness()  cut at every boundary
                    └─▶ broadcast_assumption()
                              │
   lifecycle.py ──┬───────────┘  P_BEFORE_PRIME is DERIVED from the harness
                  ├─▶ Connection: ten ordering rules
                  ├─▶ cancel()   notification, no id, never on initialize
                  ├─▶ duplicate_effects()  four policies x 1,000 calls
                  └─▶ timeout_fires_at / pool_exhaustion
   report.py ─────▶ every figure below      test_ch38.py ──▶ 37 tests
```

Folder structure. `ch38/` carries forward `roles.py`, `blast.py`, `hostmatrix.py`, `selection.py`, `affordance.py`, `mcpcat.py`, `mcpconn.py`, `costmodel.py` and the rest of Chapter 37's set — keep them in sync — and adds `stdioframe.py`, `httpwire.py`, `resume.py`, `lifecycle.py`, `report.py` and `test_ch38.py`.

Deployment and evaluation. `python3 report.py` prints every figure in about a second. `python3 -m pytest -q` runs **37 tests**, all passing from a clean restore.

Provenance. Every rule is `DOC`, quoted from specification revision 2025-11-25 with the date it was read. The polluted stdout corpus, the twelve-event stream and the simulation parameters are `BUILT`, and the random seed is fixed so the published figures are exact.

38.17 STDIO: One Skipped Line, One Lost Answer

An eight-message session with four things a real toolchain prints to `stdout` without meaning to.

```
lines on stdout                11
messages a lenient reader got  7
lines it silently skipped      4
responses expected / recovered 4 / 3
responses LOST with no error   [3]
a strict reader stops at:      line 1 is not JSON: Expecting value:
                               line 1 column 1 (char 0)
the same session, indent=2:    57 lines, 0 messages, 57 skipped
```

Read the middle three lines together. The lenient reader skipped four lines and reported nothing, because that is what lenient means. Three of the four were harmless — an `npm warn`, a `DeprecationWarning`, a Python dict printed with single quotes. The fourth was different not because it was noisier but because it was written **without a trailing newline**, so the next message landed on the same line and was destroyed with it.

That message was the response to request id 3. It is now gone. No exception was raised anywhere, no error was logged, and the client is waiting for an answer that was delivered and thrown away. It will notice in thirty seconds, or in ten minutes, or never — depending entirely on §38.22's timeout settings.

The strict reader stops at line 1. That is inconvenient and it is correct: the specification's rule is a MUST, and a transport that has already been violated is not a place to guess. A client that refuses immediately produces a bug report against the server on the first run. A client that skips produces an intermittent hang three weeks later.

And then the pretty-printer. The same eight messages, serialised with `indent=2`, become **57 lines of valid JSON and zero readable messages**. Every one of them is well-formed. Not one of them is on the

wire correctly.

COMMON MISTAKE

Debugging a STDIO server with `print()`

Symptom. A server works when run by hand and fails, intermittently, under a client. **Cause.** Anything on `stdout` that is not a message breaks framing. `print()`, progress bars and package-manager notices all land there. **Fix.** Every diagnostic goes to `stderr`. The specification says explicitly that a client SHOULD NOT treat `stderr` output as an error condition, so you are not misusing the channel. **Detection.** Run the server, pipe `stdout` through a strict JSON-per-line parser, and fail the build on the first unparseable line.

38.18 Streamable HTTP: Fifteen Outcomes, No Socket

```
  POST initialize                           200
  POST notification with a session          202
  POST request with a session               200
  POST with no session header               400
  POST with an unknown session              404
  POST with an unsupported protocol version 400
  POST with a hostile Origin                403
  POST with an incomplete Accept header     client-bug
  POST a JSON-RPC batch                     UNKNOWN
  GET to open a server stream               200
  GET with the wrong Accept header          client-bug
  DELETE to end the session                 204
  POST after the session was deleted        404
  GET where the server offers no stream     405
  DELETE where the server forbids it        405
15 of 15 reproduced
no MCP-Protocol-Version header -> ('assumed', '2025-03-26')
```

Fifteen rows, three of which are not status codes at all.

The two **client-bug** rows are the `Accept` header requirements. There is no status code for these because a conforming client never sends them; the failure belongs in your own process, at the point of construction, and it gets its own exception type for exactly that reason.

The **UNKNOWN** row is a JSON-RPC array. The transport section of revision 2025-11-25 says the POST body *"MUST be a single JSON-RPC request, notification, or response"*, and describes no batching anywhere. Documented in neither direction, so Chapter 14's rule applies: it raises rather than defaulting to accepted or rejected.

Two pairs are worth reading together. **400 and 404** both mean your session is not usable, and they mean opposite things about what to do: 400 says you forgot the header, 404 says the session is gone and you must initialize again. And **the version header's two failure modes are asymmetric** — a *wrong* version is a 400, a *missing* one is a silent downgrade to `2025-03-26`. The louder failure is the safer one.

38.19 Lifecycle Ordering: Ten Rules

```
10 of 10 behave as documented
```

```
   tools/list before initialize                rejected
   ping before initialize                      rejected
   a second initialize                         rejected
   tools/list while initialize is outstanding  rejected
   ping while initialize is outstanding        accepted
   server sampling before initialized          rejected
   server logging before initialized           accepted
   initialized with no initialize              rejected
   cancelling initialize                       rejected
   cancelling tools/call                       accepted
```

The two `accepted` rows in the middle are the whole reason to build this table. A ping may cross an outstanding `initialize`, and the server may log before it has received `notifications/initialized`. Everything else in that window is refused. Those exceptions exist so that liveness checking and diagnostics work during the one phase where nothing else does — which is precisely the phase where you most need them.

The last two rows are the cancellation rules: `initialize` **MUST NOT** be cancelled by clients, and everything else may be. A cancellation of `initialize` is not merely useless; the connection has no agreed protocol version yet, so there is no defined behaviour to fall back to.

38.20 Resumability: Fourteen Cut Points

Chapter 17 cut a Messages API stream at 93 points and found that a strict reader refused 87 of them while a naive one refused none. Same instrument, different transport, and this time there is a documented way back.

```
ids attached (SHOULD)         cuts 14  fully recovered 13  lost  12  refused  0
no ids attached (MAY)         cuts 14  fully recovered  1  lost  90  refused  0
resumed on the wrong stream   cuts 14  fully recovered  1  lost  90  refused 13
the one unrecoverable cut: at 0 events received, Last-Event-ID None,
   12 messages lost
```

With ids, thirteen of fourteen cut points recover everything. Cut the stream after the priming event, after event one, after event seven, after event eleven — reconnect with `Last-Event-ID` and the server replays the remainder. Nothing is lost and nothing is repeated.

The fourteenth is the cut before anything arrived. No event, no id, nothing to resume from, twelve messages lost. This is why the specification says the server SHOULD send a priming event *immediately*: it shrinks that window to the width of one round trip. It cannot close it, and no protocol can, because there is a moment before the first byte in which the client and the server disagree about whether the request exists at all.

Without ids, one of fourteen. The single "success" is the trivial cut where the stream had already finished. A server that does not attach ids has opted out of resumption entirely, and its clients are back to reissuing.

Across streams, refused thirteen times. The MUST NOT holds. Every cut point that had an id to quote had it rejected when the id belonged to another stream, which is the correct outcome and the one a permissive implementation would get wrong in the most damaging possible way: replaying another stream's messages into this one is the shape of the session-hijack injection documented in the security

guidance.

38.21 Disconnection Is Not Cancellation

One thousand tool calls with a side effect, twenty per cent of which lose their connection before the response arrives. Four policies, one seeded run.

```
policy                                    drops  dups   per 1,000
naive                                     210    210    210.00
resume                                    211    17     17.00
cancel-then-retry                         211    67     67.00
idempotent-effect                         210    0      0.00
resume, but the server attached no ids    210    210    210.00
residual for 'resume' is derived, not assumed: 0.0714
```

Naive: 210 per 1,000. Every disconnection becomes a reissued request, the server never stopped, and every reissue is a second effect. This is the refund in §38's introduction, and it is **6.6 times** Chapter 34's turn-layer figure of 31.65 per 1,000 — not because transport retries are more dangerous per event, but because they fire on network conditions rather than on errors an application saw.

Cancel-then-retry: 67 per 1,000. Sending `notifications/cancelled` before retrying removes two thirds of the damage and leaves a third. The residual is the documented race: cancellation *"may arrive after request processing has completed"*, and receivers *"MAY ignore"* it. There is no acknowledgement to wait for, because a notification has no id. This is the number to quote at anyone who believes an explicit cancellation makes a retry safe.

Resume: 17 per 1,000. Reconnecting instead of reissuing removes almost everything, and what remains is precisely the unrecoverable cut point from §38.20 — one drop in fourteen happens before any event arrived. The figure is derived from the harness rather than chosen, which is why it is 17 and not a round number.

Idempotent effect: 0.00. Chapter 34's loop-minted key, applied to the effect rather than to the request. It is the only row that reaches zero, and it reaches zero under every transport condition, including the one that resumption structurally cannot reach.

And the row at the bottom. A resuming client talking to a server that attached no event ids is byte-for-byte a naive client: 210 per 1,000. Your safety depends on a SHOULD in someone else's implementation. Check for ids on the first stream of every session and record their absence as a property of that server.

KEY TAKEAWAYS

The ordering to remember

Idempotency first, resumption second, cancellation third, reissue never. The first is the only one that works when everything else has failed, and it is the only one entirely within your control.

38.22 Timeouts, and the Pool

```
reset on progress, no maximum   never fired within 43200 seconds
reset on progress, maximum 300s maximum timeout at 300s
pool of 32, 4 hung requests a minute:
   no maximum   -> exhausted after 8 minutes
   maximum 300s -> steady state 20 of 32 slots held
```

A thirty-second idle timeout that resets on progress, against a server emitting progress every five seconds, does not fire in twelve hours of simulated time. It is not a long timeout. It is no timeout, dressed as one, and a server does not have to be malicious to trigger it — a loop that is stuck but still counting will do.

The consequence is a pool, not a request. Four hung requests a minute against 32 slots exhausts the pool in **8 minutes**, at which point every user of that client is affected by one misbehaving server. With a 300-second maximum the same arrival rate settles at **20 of 32** slots held: uncomfortable, visible in a metric, and survivable.

This is Chapter 29's lesson in a new place. There the finding was that `max_retries` bounds a request and only a budget bounds a fleet. Here it is that an idle timeout bounds a quiet request and only a maximum bounds a chatty one. In both cases the per-request control is the one the library gives you by default and the fleet-level control is the one you have to add.

38.23 Hands-on Labs

Lab 1 — Break your own server's stdout. Add a single `print("starting")` to a STDIO MCP server and run it under a client. *Success criterion:* you can describe exactly what the client did — refuse, hang, or silently skip — and you have changed the line to write to `stderr`.

Lab 2 — Reproduce the fifteen HTTP outcomes. Run `outcomes()` and then construct a sixteenth: a POST whose body is a response rather than a request. *Success criterion:* you predict 202 before running it, and you can quote the sentence that says so.

Lab 3 — Find the unrecoverable cut. Run the resumability harness and print every row. *Success criterion:* you can say which cut point loses messages, why, and what the priming event does about it.

Lab 4 — Measure your own duplicate rate. Take your real disconnection rate from your load balancer's metrics, substitute it for `P_DISCONNECT`, and run all four policies. *Success criterion:* a number of duplicated effects per 1,000 for your current client behaviour, and the same number for the policy you intend to move to.

Lab 5 — Bound a timeout. Find the timeout configuration in your MCP client. *Success criterion:* you can name both the idle value and the maximum, or you have discovered that there is no maximum and added one, with the pool arithmetic that justifies the number.

38.24 Lab Solutions

Lab 1. All three outcomes are conforming client behaviour, which is the point: the specification tells the *server* not to do this and leaves the client's response open. The strict reader in `stdioframe.py` raises at line 1; the lenient one skips and carries on. Moving the line to `stderr` is correct and complete — the client

MAY capture, forward or ignore it, and SHOULD NOT read it as an error.

Lab 2. 202 Accepted with no body. *"If the input is a JSON-RPC response or notification: If the server accepts the input, the server MUST return HTTP status code 202 Accepted with no body."* A response is something the client sends when the *server* made a request — a `sampling/createMessage` answer, for instance — and there is nothing for the server to reply to.

Lab 3. Cut point 0, before the priming event. `last_event_id` returns `None`, `replay` returns nothing, and all twelve messages are lost:

```
def last_event_id(received):
    for event in reversed(received):
        if event["id"] is not None:
            return event["id"]
    return None
```

The priming event does not eliminate this case; it shrinks it to the interval between the request leaving and the first event arriving. That interval is where §38.21's residual 17 per 1,000 lives.

Lab 4. `duplicate_effects` takes the rate as a module constant; parameterise it and sweep. The shape of the answer does not change with the rate — `naive` scales linearly with disconnections, `idempotent-effect` stays at zero — which is itself the argument for fixing the effect rather than tuning the transport.

Lab 5. Most clients expose one timeout. If yours does, the maximum is the one you are missing. Size it from the pool: with p slots and h hung requests per minute, a maximum of T seconds holds `h × T / 60` slots in steady state, and you want that comfortably under p. At 32 slots and 4 per minute, 300 seconds gives 20 — the figure in §38.22.

38.25 Best Practices

BEST PRACTICE

Nothing but messages on `stdout`

Route every diagnostic to `stderr`, including the ones your framework emits for you. Test it by parsing your own server's `stdout` strictly in CI.

BEST PRACTICE

Attach event ids to every SSE event, and send a priming event

Ids are a SHOULD and they are the difference between 17 and 210 duplicated effects per 1,000. Send the empty-data priming event immediately, so a client that drops early still has something to quote.

BEST PRACTICE

Resume, do not reissue

On a disconnection, reconnect with `Last-Event-ID`. Reserve reissuing for the case where you have no id at all — and make that case safe with an idempotency key rather than with hope.

BEST PRACTICE

Send cancellations; do not trust them

`notifications/cancelled` frees the server's resources in the common case and is the right thing to send when a user presses stop. Log the reason, stop waiting, and ignore any response that arrives afterwards. Never treat it as confirmation.

BEST PRACTICE

Set two timeouts and justify the second one

An idle timeout that may reset on progress, and a maximum that may not. Derive the maximum from your pool size, not from a default.

BEST PRACTICE

Treat a session 404 as an instruction, not an error

It means initialize again. Surface it as a failure only if the fresh initialize also fails.

BEST PRACTICE

Send `MCP-Protocol-Version` on every HTTP request

Absent, you are silently speaking `2025-03-26`. There is no error to alert you.

BEST PRACTICE

Never authenticate with a session id

Verify every inbound request. Bind session ids to a user identity — `<user_id>:<session_id>` — and generate them with a secure random source.

38.26 Common Mistakes

COMMON MISTAKE

Treating a dropped connection as a cancelled request

Symptom. Duplicated side effects that correlate with network conditions rather than with load. **Cause.** *"Disconnection SHOULD NOT be interpreted as the client cancelling its request."* **Fix.** Resume with `Last-Event-ID`; make the effect idempotent. **Detection.** 210 duplicated effects per 1,000 at a 20 per cent drop rate — measure yours by counting effects against approved requests.

COMMON MISTAKE

Believing a cancellation notification stopped anything

Symptom. A retry after an explicit cancel still duplicates work, occasionally. **Cause.** Cancellation is a notification with no id, may arrive after completion, and MAY be ignored. **Fix.** Treat it as best-effort resource cleanup, never as a precondition for retrying. **Detection.** 67 per 1,000 in $38.21.

COMMON MISTAKE

Logging to `stdout` from a STDIO server

Symptom. Intermittent failures under a client, never when run by hand. **Cause.** `stdout` is the wire. **Fix.** `stderr`. **Detection.** Parse your own `stdout` strictly in CI.

COMMON MISTAKE

Pretty-printing JSON-RPC

Symptom. Nothing works at all, in a way that looks like a parser bug. **Cause.** `indent=2` splits every message across lines. 57 lines, zero messages. **Fix.** Compact separators, one line per message. **Detection.** Count lines against messages. They must be equal.

COMMON MISTAKE

An idle timeout with no maximum

Symptom. A pool exhausts under a server that is responsive but never finishes. **Cause.** Resetting the clock on progress with no ceiling means no timeout at all. **Fix.** Add a maximum, sized from the pool. **Detection.** 8 minutes to exhaust 32 slots at 4 hung requests a minute.

COMMON MISTAKE

Assuming SSE messages are broadcast to every open stream

Symptom. Missing notifications, or a de-duplicator that discards real messages. **Cause.** The server MUST NOT send the same message on two streams. **Fix.** Treat each stream as carrying a disjoint subset. **Detection.** Count messages across all open streams; the total must equal the server's count, with no overlap.

COMMON MISTAKE

Reusing a JSON-RPC id to retry

Symptom. Undefined behaviour, or a server that answers the wrong question. **Cause.** A request id MUST NOT have been used before in the same session. **Fix.** New id per request; correlate at your own layer with your idempotency key. **Detection.** Assert uniqueness of outbound ids per session in your client.

38.27 Performance Considerations

Framing is not free, but it is not the cost. Compact separators shave a few per cent off a stdio stream. The real transport cost is round trips, and Chapter 35 already established that the number that matters is *sampling* round trips rather than HTTP ones.

Long-lived streams versus polling. A server may close the connection while keeping the stream alive, having sent a `retry` field the client MUST respect. That is the documented way to avoid holding thousands of sockets, and it turns a held-connection cost into a polling cost you can tune. Respect the `retry` value rather than choosing your own; it is the server telling you what it can afford.

Resumption is cheaper than reissuing in every dimension. A replay re-sends messages the server already generated. A reissue re-executes the work. The token cost, the latency and the side effects are all on the second one.

Sessions amortise initialization. One handshake per session, not per call. This is why the 404-means-start-again rule is worth handling automatically: a client that treats it as an error pays a full re-initialization *and* a user-visible failure, when it could have paid only the first.

38.28 Security Considerations

Four items belong to the transport layer specifically. Chapter 40 takes the authorization framework properly; these are the ones you cannot defer.

Origin validation and localhost binding. A Streamable HTTP server **MUST** validate the `Origin` header on all incoming connections and respond **403** if it is present and invalid, and when running locally it **SHOULD** bind to `127.0.0.1` rather than `0.0.0.0`. Without both, *"attackers could use DNS rebinding to interact with local MCP servers from remote websites"*. If you run a local HTTP MCP server, check these today.

Session hijacking. Two attacks are documented. In the impersonation variant an attacker who obtains a session id makes calls as the legitimate user. In the injection variant an attacker sends a malicious event to a second server instance with the same session id; it is enqueued, the first instance polls it, and it reaches the client as an asynchronous or *resumed* response — the resumability mechanism of §38.20 turned into a delivery channel. The mitigations are explicit: servers implementing authorization **MUST** verify all inbound requests, **MUST NOT** use sessions for authentication, **MUST** use secure non-deterministic session ids, and **SHOULD** bind them to user-specific information with a key like `<user_id>:<session_id>`.

Local server compromise. A STDIO server is a binary executing on the user's machine, and the guidance names the consequences plainly — arbitrary code execution with the client's privileges, no visibility into what is being run, command obfuscation. A client offering one-click local server configuration **MUST** show the exact command without truncation and require explicit approval, and **SHOULD** sandbox the spawned process. Chapter 37 measured why: six of nine asset classes reachable, against one for a remote server.

Token passthrough is forbidden. An MCP server **MUST NOT** accept tokens that were not explicitly issued for it. Chapter 40 covers the authorization framework; the transport-level version of the rule is that a bearer you send to a server is a credential *for that server*, and a server that forwards it downstream has broken the audience separation that makes revocation and auditing possible.

38.29 Real Production Examples

A payments integration over Streamable HTTP. The client resumes rather than reissues, and every effect carries a loop-minted idempotency key. The team's post-incident rule after their own duplicate-refund episode was one line in the client: a reissue is only permitted when `Last-Event-ID` is absent, and it must carry the original key. Duplicated effects went from a monthly incident to none.

A local database server over stdio. The team's CI runs the server for five seconds, captures `stdout`, and fails the build if any line is not a parseable JSON-RPC message. The check has caught a dependency's install notice twice and a debug `print()` once, each time before it reached a developer's machine.

A long-running indexing tool. The server emits progress every few seconds. The client resets its idle timeout on progress and enforces a 600-second maximum, chosen from a pool of 64 and an observed rate of six stuck requests an hour. Before the maximum existed, a single stuck index job took the assistant offline for everyone in the workspace.

38.30 Certification Notes

Concept	Exam domain	Weight
The three JSON-RPC shapes and their id rules	D2 Tool Design & MCP Integration	18%
STDIO framing and the `stdout` MUST NOT	D2	18%
Streamable HTTP status codes and `Accept`	D2	18%
Session lifecycle and the 404 that means restart	D2 · D5 Context Management & Reliability	18% · 15%
Resumability and `Last-Event-ID`	D5 · D2	15% · 18%
Disconnection versus cancellation	D5 · D1 Agentic Architecture	15% · 27%
Idempotency across a transport retry	D5 · D1	15% · 27%
Timeouts, progress and the maximum	D5	15%
The protocol version header's silent default	D2	18%
Origin validation and session hijacking	D5 · D2	15% · 18%

CERTIFICATION NOTE

The three questions that recur

One: what may you conclude from a dropped connection? Nothing about the server's work. Any answer involving an automatic retry of a side-effecting call is wrong.

Two: which status code means what. 202 for a notification or response, 400 for a missing session header or a bad version, 403 for a bad Origin, 404 for a terminated session — and 404 means *initialize again*, not *fail*. 405 appears twice, for a GET where no stream is offered and a DELETE the server forbids.

Three: which rules are MUST and which are SHOULD. Event ids are a SHOULD, which is why you must check for them. The maximum timeout is a SHOULD, which is why most clients do not have one. Cross-stream replay is a MUST NOT, and cancelling `initialize` is a MUST NOT.

38.31 Twenty Practice Questions

1. Which three message shapes does MCP define, and which one must not carry an id?
2. May a JSON-RPC id be `null` in MCP?
3. What must a server never write to `stdout`, and where should it write it instead?
4. What should a client conclude from output on a server's `stderr`?
5. Which two content types must a POST's `Accept` header list, and why both?
6. What status code does a server return for a POST whose body is a notification?
7. Two content types are permitted in response to a POST carrying a request. Name them and say who chooses.
8. What does a 404 on an established session mean, and what must the client do?
9. When does a server return 405 on a GET?
10. What must a server do with an invalid `Origin` header, and what attack does that prevent?
11. What does a server assume if `MCP-Protocol-Version` is missing? What if it is present but unsupported?
12. What must be true of an SSE event id if the server attaches one?
13. Which header does a client send to resume, and what is the server permitted to replay?
14. May a server replay a stream's messages onto a different stream?
15. Why does the specification recommend an immediate priming event?
16. Is a disconnection a cancellation? Quote the rule.
17. Name three documented reasons a receiver may ignore a cancellation notification.
18. Which single request must never be cancelled by a client?
19. Under what condition does resetting a timeout on progress leave you with no timeout?
20. Why must a session id not be used for authentication?

38.32 Answers

1. **Request, response and notification.** A **notification** MUST NOT carry an id. Distractors offering "batch" describe a JSON-RPC feature the 2025-11-25 transport section does not describe — §38.18 records it as UNKNOWN.
2. **No.** *"Unlike base JSON-RPC, the ID MUST NOT be* `null`*."* The trap is that plain JSON-RPC 2.0 permits it.
3. **Anything that is not a valid MCP message.** Logs go to `stderr`.
4. **Nothing in particular.** The client *"SHOULD NOT assume* `stderr` *output indicates error conditions"*. Treating it as failure is the common mistake.
5. `application/json` **and** `text/event-stream`**.** Both, because the *server* chooses which to return and the client MUST support either.
6. **202 Accepted, with no body.** Not 200, and not 204.
7. `application/json` with a single JSON object, or `text/event-stream` opening a stream. The **server** chooses.
8. **The session has been terminated.** The client MUST start a new session by sending a fresh `InitializeRequest` with no session id. "Report an error to the user" is the tempting wrong answer.
9. When the server **does not offer an SSE stream at that endpoint**. The same code also appears on a DELETE the server does not permit.
10. **Respond 403 Forbidden.** It prevents **DNS rebinding**, where a web page you visited drives an MCP server on your own machine.
11. Missing → assume `2025-03-26`, silently. Present but unsupported → **400 Bad Request**. The asymmetry is the exam point.
12. It **MUST be globally unique across all streams within the session**, and it **SHOULD** encode enough information to identify its own stream.
13. `Last-Event-ID`**.** The server MAY replay messages that would have been sent after that id, **on the stream that was disconnected**.
14. **No — MUST NOT.** This is the rule that stops a resumption from becoming an injection channel.
15. So a client that disconnects before any real message still has an id to resume from. It shrinks the unrecoverable window; it cannot remove it.
16. **No.** *"Disconnection SHOULD NOT be interpreted as the client cancelling its request."*
17. The referenced request is **unknown**, processing has **already completed**, or the request **cannot be cancelled**.
18. `initialize`**.**
19. When the server emits progress **more often than the idle interval** and no **maximum** is enforced. Twelve simulated hours produced no timeout in §38.22.
20. Because an attacker who obtains or guesses the id would then be authorised. Servers **MUST** verify all inbound requests and **MUST NOT** use sessions for authentication; ids should be non-deterministic and bound to a user identity.

38.33 Ten Practical Exercises

1. Pipe one of your STDIO servers' `stdout` through a strict parser for a full session and count unparseable lines.
2. Add the same check to CI as a failing test.
3. Capture one Streamable HTTP session and list every status code you observe against §38.18's table.
4. Confirm your client sends `MCP-Protocol-Version` on every request after initialization.
5. Find out whether your servers attach SSE event ids. Record the answer per server.
6. Trace what your client does on a mid-stream disconnection. Resume, reissue or fail?
7. Count the side-effecting tools in your catalogue and check which carry an idempotency key.
8. Find the idle timeout and the maximum in your client configuration. Add the second if it is missing.
9. Compute your pool's steady-state occupancy at your observed rate of stuck requests.
10. Check whether any local HTTP MCP server you run binds to `0.0.0.0` and validates `Origin`.

38.34 Five Challenge Labs

1. **A resumption-aware client.** Implement `Last-Event-ID` resumption end to end, including the case where no id was received, and prove with the harness that your duplicate rate matches §38.21's `resume` row.
2. **A framing fuzzer.** Generate stdout streams with pollution at every position and measure, for each position, whether the loss is a log line or a response. Publish the distribution.
3. **Cancellation under race.** Simulate cancellations arriving uniformly across the request's lifetime and plot duplicated effects against arrival time. Identify the point after which cancelling is pure ceremony.
4. **A session store that resists hijacking.** Implement `<user_id>:<session_id>` binding and demonstrate that a guessed id belonging to another user is rejected.
5. **Backwards compatibility.** Implement the documented probe — POST `InitializeRequest`, fall back to GET on 400, 404 or 405 — and write the test that distinguishes a legacy server from a terminated session, since both can answer 404.

38.35 Ten Interview Questions

1. A tool call's connection drops. What do you know about the server's work?
2. Why can a cancellation not be confirmed?
3. What does an event id buy you, and what does its absence cost?
4. Why must a server never replay one stream's messages onto another?
5. Why is `stdout` pollution such a common failure in STDIO servers?
6. When is 404 an instruction rather than an error?
7. What is wrong with resetting a timeout every time progress arrives?
8. How would you make a side-effecting MCP tool safe to retry?
9. Why is a missing protocol version header more dangerous than an invalid one?
10. How would you detect, from the outside, that a client is reissuing rather than resuming?

38.36 Summary

MCP's two transports differ in almost every detail and agree on the thing that matters: the wire is a sequence of JSON-RPC messages, and a break in it tells you about the wire. Over **stdio** the server is a subprocess and `stdout` *is* the wire, so anything else written there — a `print()`, a package-manager notice, a progress bar — corrupts it. Chapter 38 measured both possible client responses over an eight-message session polluted with four ordinary lines: a strict reader stops at line one, and a lenient reader recovers seven messages, skips four lines, and silently loses the response to request 3 because one polluted line arrived without a trailing newline and swallowed the message behind it. The same session pretty-printed produced **57 lines of valid JSON and zero readable messages**.

Over **Streamable HTTP** there is one endpoint, POST and GET, and a table of documented outcomes that the mini project reproduces in full: 202 for a notification, 200 with either JSON or an event stream for a request, 400 for a missing session header or a bad protocol version, 403 for a bad `Origin`, 404 for a terminated session, and 405 twice — for a GET where no stream is offered and a DELETE the server forbids. Fifteen of fifteen reproduced offline, plus two client-side MUSTs that have no status code because a conforming client never sends them, and one UNKNOWN that raises. Two asymmetries are worth carrying: 400 and 404 both mean the session is unusable but demand opposite responses, and a *wrong* protocol version is a loud 400 while a *missing* one is a silent downgrade to `2025-03-26`.

The lifecycle is initialization, operation, shutdown, and its ordering rules reproduce exactly — ten of ten — including the two deliberate exceptions that let a ping cross an outstanding `initialize` and let a server log before it has been told the client is ready. Shutdown has no messages at all, only the transport: close `stdin`, then `SIGTERM`, then `SIGKILL`, or close the connection and `DELETE` the session.

Then the chapter's argument. Cutting a twelve-event stream at every one of its fourteen boundaries, a client that resumes with `Last-Event-ID` recovers everything at **13 of 14** cut points; without event ids, **1 of 14**; and a replay quoted from another stream is refused at all thirteen, as the MUST NOT requires. The single unrecoverable cut is the drop that happens before any event has arrived, and it is why the specification tells servers to send a priming event immediately — it shrinks that window without closing it. Feeding that fraction into a thousand side-effecting calls at a twenty per cent disconnection rate gives the four numbers this chapter exists for: naive reissue **210** duplicated effects per 1,000, explicit cancellation followed by a retry **67**, resumption **17**, and an idempotent effect **0.00**. A resuming client talking to a server that attached no ids scores **210** — identical to the naive one, because the SHOULD it depends on belongs to somebody else.

Finally, timeouts. Resetting the clock on progress is reasonable and it is not a timeout: against progress every five seconds, a thirty-second idle limit did not fire in twelve simulated hours, and a pool of 32 was exhausted in **8 minutes** at four hung requests a minute. A 300-second maximum settles the same load at **20 of 32** slots. The pattern is Chapter 29's, one layer down: the per-request control ships in the library and the fleet-level control is the one you have to add yourself.

KEY TAKEAWAYS

1. Three message shapes. A notification has no id, gets no response, and therefore can never be confirmed.
2. A request id MUST NOT be null and MUST NOT be reused within a session. Retrying by reusing an id is a protocol violation.
3. Over stdio, `stdout` is the wire and `stderr` is for logs — and a client SHOULD NOT read `stderr` as an error.
4. Pretty-printing destroys framing: 57 lines, 0 messages.
5. A lenient reader that skips unparseable lines will eventually skip a response, and nothing anywhere will report it.
6. 202 for a notification; 400 for a missing session id or bad version; 403 for a bad `Origin`; 404 means **initialize again**; 405 for an unoffered GET stream or a forbidden DELETE.
7. A missing `MCP-Protocol-Version` header is a silent downgrade to `2025-03-26`; an invalid one is a 400.
8. Event ids are a SHOULD and they are worth **210 → 17** duplicated effects per 1,000. Check every server for them.
9. **A disconnection is not a cancellation**, and an explicit cancellation is not confirmation — cancel-then-retry still duplicated **67 per 1,000**.
10. Only an idempotent effect reaches **0.00**, under every transport condition, including the one resumption cannot reach.

38.37 Glossary

Term	Meaning
Transport	The mechanism carrying JSON-RPC messages. MCP defines stdio and Streamable HTTP; custom transports are permitted.
stdio transport	The client launches the server as a subprocess; messages are newline-delimited on `stdin`/`stdout`; logs go to `stderr`.
Streamable HTTP	One endpoint supporting POST and GET, optionally upgrading a response to an SSE stream.
Framing	How a receiver finds message boundaries. A newline over stdio; a body or an SSE event over HTTP.
Request	`jsonrpc` + `id` + `method`. The id must be a string or integer, never null, never reused in a session.
Response	The same `id` plus exactly one of `result` or `error`.
Notification	`jsonrpc` + `method`, no id, no response.
`MCP-Session-Id`	An optional header assigned at initialization and required on every later request; visible ASCII only.
Session 404	The session is gone. The client MUST start a new one with a fresh `InitializeRequest` and no id.

Term	Meaning
`MCP-Protocol-Version`	Required on every HTTP request after initialization. Absent → the server assumes `2025-03-26`. Invalid → 400.
Event id	An optional SSE `id`, globally unique within the session, ideally encoding its own stream.
`Last-Event-ID`	The header a client sends on a resumption GET, naming the last event it received.
Resumption	Replaying messages after a given event id, on the disconnected stream only.
Priming event	An immediate SSE event with an id and empty data, so an early disconnection still has something to resume from.
`retry` field	An SSE field telling the client how long to wait before reconnecting. The client MUST respect it.
`notifications/cancelled`	A cancellation request carrying `requestId` and an optional `reason`. Best effort; may be ignored; may arrive too late.
`progressToken`	Placed by the requester in `params._meta`; quoted back in `notifications/progress`, whose `progress` must increase.
Idle timeout	A limit on silence, which may reset on progress.
Maximum timeout	A limit on total duration, which may not. A SHOULD, and the one most clients omit.
DNS rebinding	The attack that `Origin` validation and localhost binding exist to prevent.
Session hijacking	Obtaining a session id to impersonate a client, or to inject events a legitimate client then receives.

38.38 Further Reading

Primary sources. Specification revision **2025-11-25** at `modelcontextprotocol.io/specification/2025-11-25`: `basic` for the JSON-RPC message types and the `_meta` conventions, `basic/transports` for both transports and their security warnings, `basic/lifecycle` for initialization, timeouts and shutdown, and `basic/utilities/cancellation` and `basic/utilities/progress` for the two notifications this chapter measures. The `basic/security_best_practices` page covers session hijacking, token passthrough, SSRF during OAuth discovery and local server compromise, and is the right thing to read immediately before Chapter 40. For the older transport, `specification/2024-11-05/basic/transports#http-with-sse` documents what the compatibility probe in §38.11 is looking for.

Where to go next in this book. Chapter 39 moves up from the wire to the server's primitives — resources, tools and prompts as protocol objects, with the subscription and `listChanged` machinery this chapter only named. **Chapter 40** builds and secures a server, including the MCP authorization framework and the attacks summarised in §38.28. **Chapter 41** opens Part 8 with Claude Code, which speaks four of these transports and reconnects with the exponential backoff Chapter 29 would recognise. For tracing a request across a transport boundary, **Chapter 57**; for what a hostile `tool_result` can do once it has crossed one, **Chapter 58**.